

**AGENTFLOW WEB3: A NO-CODE, AI-ASSISTED
PLATFORM FOR COMPOSING AND HOSTING
DECENTRALIZED APPLICATIONS**

A PROJECT REPORT

Submitted By

NAGIPRAGALATHAN N (110824405007)

In partial fulfilment for the award of the degree

Of

MASTER OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

JAYA ENGINEERING COLLEGE, THIRUNINRAVUR

ANNA UNIVERSITY : CHENNAI 600 025

JULY 2026

ANNA UNIVERSITY : CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report “**AGENTFLOW WEB3: A NO-CODE, AI-ASSISTED PLATFORM FOR COMPOSING AND HOSTING DECENTRALIZED APPLICATIONS**” is the bonafide work of **NAGIPRAGALATHAN N (110824405007)** who carried out the project work under my supervision.

Mr. J. LIN EBY CHANDRA

Head of the Department,
Department of CSE,
Jaya Engineering College,
Thiruninravur – 602 024.

Mr. J. LIN EBY CHANDRA

Supervisor,
Department of CSE,
Jaya Engineering College,
Thiruninravur – 602 024.

ANNA UNIVERSITY : CHENNAI 600 025

VIVA-VOCE EXAMINATION

The viva-voce examination of the project work titled “**AGENTFLOW WEB3: A NO-CODE, AI-ASSISTED PLATFORM FOR COMPOSING AND HOSTING DECENTRALIZED APPLICATIONS**” submitted by **NAGIPRAGALATHAN N (110824405007)** is held on _____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ABSTRACT

AgentFlow Web3 is a no-code, AI-assisted platform that lets anyone compose, run, and host decentralized applications without writing code. Building on a blockchain normally demands smart contract programming, careful key handling, low-level calls to blockchain nodes, and a separately built web interface, which keeps most useful Web3 ideas out of reach. The platform addresses this gap by making every on-chain action a first-class building block on a visual canvas that anyone can wire together with logic and artificial intelligence, attach to a drag-and-drop interface, and publish at a shareable link. A topological execution engine compiles a workflow into a dependency graph and runs deterministic on-chain primitives, conditional logic, and large language model agents in a single ordered pass, passing data between blocks through typed input and output handles. The on-chain primitives — wallet, balance, contract read and write, transfer, message signing, event queries, price feeds, chain state, and name resolution — are implemented over the viem Ethereum client, while agent blocks use a vendor-neutral interface so several model providers are interchangeable. An interface builder binds page elements to workflows and serves the result at a public per-page link, and an assistant supports authoring in plain language. Implemented as a single Next.js and TypeScript application with PostgreSQL storage through Prisma, the platform runs real on-chain workflows. Measured results show that the orchestration layer adds only a few milliseconds per workflow and that end-to-end time is set by the blockchain endpoint rather than by the platform, confirming that raising the level of abstraction does not make the resulting applications slow. Because the building blocks are general, the same canvas expresses a wide space of applications — from decentralized finance and trading to creator tools, organizations, payments, monitoring, identity, and autonomous agents.

TABLE OF CONTENTS

	PAGE NO.
ABSTRACT	iv
LIST OF TABLES	vii
LIST OF FIGURES	viii
LIST OF SYMBOLS, ABBREVIATIONS & DEFINITIONS	ix
1. INTRODUCTION	2
1.1 SYSTEM OVERVIEW	2
1.2 OBJECTIVE	2
1.3 SYSTEM STUDY	3
1.3.1 Existing System	3
1.3.2 Literature Survey	4
1.3.3 Proposed System	4
1.4 ORGANIZATION OF THE REPORT	5
2. SOFTWARE REQUIREMENTS SPECIFICATION	7
2.1 EXTERNAL INTERFACE REQUIREMENTS	7
2.2 SYSTEM FEATURES	7
2.2.1 Visual Workflow Authoring Module	7
2.2.2 On-Chain Primitive Module	8
2.2.3 AI Agent and Orchestration Module	8
2.2.4 Interface Builder and Binding Module	8
2.2.5 Hosting and Publishing Module	8
2.2.6 Scheduling and Triggers Module	8
2.2.7 Assisted Authoring Module	8
2.3 OTHER NON-FUNCTIONAL REQUIREMENTS	9
3. SOFTWARE DESIGN	11
3.1 ARCHITECTURAL DESIGN	11
3.1.1 System Architecture	11
3.1.2 Use Case Diagram	12
3.1.3 Sequence Diagram	13
3.2 DECOMPOSITION DESCRIPTION	14
3.2.1 Workflow Execution Engine	14
3.2.2 On-Chain Primitive Engine	15
3.2.3 AI Agent Engine	15

3.2.4 Interface Binding and Hosting Engine	15
3.2.5 Scheduler	16
3.3 COMPONENT DESIGN	16
3.4 DATA DESIGN	17
3.5 HUMAN INTERFACE DESIGN	17
4. IMPLEMENTATION	20
4.1 SOFTWARE ENVIRONMENT	20
4.2 PROJECT STRUCTURE	21
4.3 FRONTEND IMPLEMENTATION	21
4.4 BACKEND IMPLEMENTATION	22
4.5 API ENDPOINTS	23
4.6 DATABASE SCHEMAS	24
5. TEST PLAN AND TESTING	27
5.1 TESTING LEVELS	27
5.2 TESTING TOOLS	27
5.3 TEST CASES	28
5.4 PERFORMANCE TESTING	30
5.5 TEST RESULT SUMMARY	31
5.6 DEFECTS AND RESOLUTIONS	32
6. RESULTS AND OBSERVATIONS	34
6.1 FUNCTIONAL VALIDATION	34
6.2 EXECUTION LATENCY	35
6.3 CAPABILITY COMPARISON	35
6.4 OBSERVATIONS	36
7. CONCLUSION AND FUTURE WORK	38
7.1 CONCLUSION	38
7.2 FUTURE WORK	38
APPENDIX	41
REFERENCES	46

LIST OF TABLES

Table No.	Table Name	Page No.
4.1	Core Platform API Endpoints	24
5.1	Testing Tools and Environments	28
5.2	Workflow Execution Latency over Seven Runs	31
5.3	Orchestration Overhead Summary	31
5.4	Test Result Summary	31
6.1	Functional Validation of On-Chain Workflows	35
6.2	Native Capability Comparison	36

LIST OF FIGURES

Figure No.	Figure Name	Page No.
3.1	AgentFlow Web3 System Architecture	12
3.2	Use Case Diagram	13
3.3	Sequence Diagram	14
3.4	Workflow Execution Model	14
3.5	Taxonomy of Composable Primitives	16
3.6	Interface Binding and Hosting Flow	15
3.7	Idea-to-dApp Lifecycle	18
3.8	Visual Workflow Builder Interface	17
3.9	Interface (Page) Builder	18
6.1	Median Workflow Execution Latency	35
A.1	Workflows Dashboard Interface	41
A.2	Visual Workflow Builder Interface	42
A.3	Interface (Page) Builder Interface	42
A.4	dApps Gallery Interface	43
A.5	Hosted dApp Page Interface	44

LIST OF SYMBOLS, ABBREVIATIONS & DEFINITIONS

SYMBOLS

- — Data / control flow between blocks
- ◇ — Decision / conditional branch
- — Building block (node)
- — Persistent data store (PostgreSQL)
- ⇄ — Bidirectional communication
- ⌘ — On-chain interaction

ABBREVIATIONS

- AI** — Artificial Intelligence
- LLM** — Large Language Model
- dApp** — Decentralized Application
- Web3** — Decentralized, user-owned web
- DAG** — Directed Acyclic Graph
- EVM** — Ethereum Virtual Machine
- RPC** — Remote Procedure Call
- ERC** — Ethereum Request for Comments (token standard)
- NFT** — Non-Fungible Token
- ENS** — Ethereum Name Service
- DeFi** — Decentralized Finance
- DAO** — Decentralized Autonomous Organization
- API** — Application Programming Interface
- UI** — User Interface
- ORM** — Object-Relational Mapping
- L2** — Layer-2 scaling network
- AA** — Account Abstraction

DEFINITIONS

No-Code Development: Building working software by visually configuring and connecting blocks instead of writing source code.

Workflow: A set of building blocks and directed connections that the engine runs in dependency order.

Building Block (Node): A typed unit of behaviour — a trigger, on-chain action, logic step, AI agent, tool, or output.

On-Chain Primitive: A native blockchain action (read or write) exposed as a building block over the viem client.

Topological Sort: An ordering of a directed acyclic graph in which every block runs only after the blocks feeding it.

Typed Handle: A named input or output port on a block through which values pass to connected blocks.

LLM Agent: A block that calls a large language model to reason, extract, classify, or converse, optionally with memory and tools.

Interface Binding: The link between a page action element (such as a button) and the workflow it runs.

Hosting: Serving a built interface at a public, shareable per-page link so anyone can use the application.

Composability: The property by which a small set of general blocks, combined freely, expresses a wide space of applications.

Smart Contract: A program deployed on a blockchain whose functions can be read or invoked by transactions.

Oracle: An external data source, such as a price feed, made available to on-chain logic.

CHAPTER 1

INTRODUCTION

1. INTRODUCTION

1.1 SYSTEM OVERVIEW

AgentFlow Web3 is a no-code, AI-assisted platform developed to make the creation of decentralized applications accessible to people who are not blockchain engineers. Web3 technology gives users ownership of data, money, and digital assets without a central authority, yet turning an idea into a working decentralized application still requires smart contract programming, careful private-key handling, low-level calls to blockchain nodes, and a separately built web interface. As a result, the space of useful Web3 ideas is far larger than the small group of engineers able to build them.

The platform closes this gap by raising the level of abstraction. Every on-chain action is treated as a first-class building block that a user can place on a visual canvas, wire together with logic and artificial intelligence, attach to a drag-and-drop interface, and publish at a shareable link — all without writing code. Because the building blocks are general, the same canvas can express a very wide range of applications, from decentralized finance automation and trading to non-fungible-token workflows, decentralized organizations, payments, on-chain monitoring, identity, and autonomous agents that read and act on chain state.

The system is implemented as a single Next.js and TypeScript application. A topological execution engine compiles each workflow into a dependency graph and runs its blocks in order; a capabilities layer provides native on-chain primitives over the viem Ethereum client and large language model agents through a vendor-neutral interface; a composition and hosting layer binds interfaces to workflows and serves them at public links; and a PostgreSQL database, accessed through Prisma, stores workflows, pages, runs, and settings. An assistant supports the user throughout, answering questions about blocks and workflows in plain language while keeping the workflow fully visible and editable.

1.2 OBJECTIVE

The primary objective of AgentFlow Web3 is to develop a no-code, AI-assisted platform capable of composing, running, and hosting decentralized applications through a single visual surface, while keeping the orchestration overhead negligible so that the

achievable speed of an application is set by the blockchain endpoint rather than by the platform. The specific objectives are:

- To make every on-chain action a configurable building block so that a user can read a balance, call a contract, transfer value, sign a message, or react to an event by placing a block and filling in a few fields.
- To provide a topological execution engine that compiles a workflow into a dependency graph, rejects cycles and unreachable blocks, and runs deterministic on-chain blocks, conditional logic, and AI agents in one ordered pass.
- To pass data between blocks through typed input and output handles so that blockchain reads and writes combine as freely as ordinary logic and artificial intelligence.
- To integrate large language model agents through a vendor-neutral interface so that several model providers are interchangeable, with optional memory and tools.
- To provide a drag-and-drop interface builder that binds page elements to workflows and hosts the result at a shareable per-page link, turning logic into a usable product.
- To support assisted authoring through an in-platform assistant that guides building without hiding the workflow.
- To validate the approach on real on-chain workflows and measure the orchestration overhead the paradigm introduces.
- To design an extensible architecture that can support future enhancements such as account abstraction, multi-network composition, and a shared marketplace of workflow and interface bundles.

1.3 SYSTEM STUDY

1.3.1 Existing System

Mainstream visual automation platforms let users express logic by connecting configurable blocks, and low-code and no-code technology now accounts for a large and growing share of new software. However, these tools expose no native blockchain actions; any on-chain interaction must be expressed through generic web-request nodes that demand manual encoding of calls, signatures, and node endpoints. On the other side, Web3 developer toolkits and contract frameworks provide powerful on-chain capability but assume a programmer who can write contracts and front-end code. Most existing

solutions therefore address either the building experience or the blockchain capability, but not both, and a builder who wants the full path from logic to a shareable product must combine several tools manually.

Furthermore, recent work that joins large language models with blockchains is aimed largely at auditing contracts or coordinating machines rather than at helping a person build. The lack of a unified, no-code surface that treats on-chain actions as native blocks, composes them with AI, and hosts the result creates fragmentation, a steep learning curve, and a high barrier to entry for non-experts.

1.3.2 Literature Survey

Visual automation has grown from a convenience into a mainstream way of building software. Node-graph tools let people express logic by connecting configurable blocks, and industry analyses report that workflow automation is among the leading uses of no-code technology. A clear recent trend is the blending of automation with artificial intelligence, which turns fixed pipelines into systems that can decide at run time.

Large language models have driven a wave of work on agents that reason and act over several steps. The ReAct method showed that interleaving reasoning with actions lets a model build and revise a plan while gathering information from outside sources. Tool use and multi-agent coordination followed, and benchmarks such as FlowBench report that explicit, flowchart-style workflow structure helps language-model agents plan most effectively — a finding that supports exposing an explicit, editable graph to the user.

The Web3 stack itself rests on well-defined standards: a general-purpose contract platform provides the execution layer, fungible and non-fungible token standards define the assets that most applications touch, oracle networks bring outside data such as price feeds on chain, and account abstraction points toward smart accounts that improve usability. Node-based visual programming and assisted website builders provide the interface side. AgentFlow Web3 draws these lines together, applying the familiar node-graph style to native on-chain actions and joining the visual interface to the visual logic in one hostable platform.

1.3.3 Proposed System

AgentFlow Web3 introduces a unified no-code, AI-assisted paradigm in which on-chain actions are native blocks on a visual canvas. A user composes a workflow by

connecting blocks, augments it with logic and AI agents, attaches it to a drag-and-drop interface, and publishes the result at a shareable link, with an assistant available throughout. A topological engine runs the workflow as a single ordered pass, so the same model expresses a simple balance check and a multi-step automation that reasons and then acts on chain.

Advantages of the proposed system over existing approaches are summarized below.

Advantages of the Proposed System:

- Native on-chain blocks remove the constant context-switching that fragments normal Web3 work.
- A small set of general primitives, combined freely, expresses a wide space of applications.
- Deterministic on-chain blocks, conditional logic, and AI agents run inside one ordered execution pass.
- An interface builder binds pages to workflows and hosts them, so logic becomes a shareable product.
- The orchestration overhead is only a few milliseconds, so the no-code abstraction is effectively free.

1.4 ORGANIZATION OF THE REPORT

The remainder of this report is organized as follows. Chapter 2 presents the software requirements specification, including external interfaces, system features, and non-functional requirements. Chapter 3 describes the software design, covering the architectural design, use-case and sequence models, the decomposition of the platform into engines, the component and data design, and the human interface design. Chapter 4 details the implementation, including the software environment, project structure, frontend and backend implementation, API endpoints, and database schemas. Chapter 5 presents the test plan and testing. Chapter 6 reports results and observations. Chapter 7 concludes the report and outlines future work, followed by an appendix of interface screenshots and the list of references.

CHAPTER 2

SOFTWARE REQUIREMENTS SPECIFICATION

2. SOFTWARE REQUIREMENTS SPECIFICATION

This chapter specifies the functional and non-functional requirements of the AgentFlow Web3 platform. It defines the external interfaces through which the system interacts with users and external services, the principal system features, and the quality attributes the platform is expected to satisfy.

2.1 EXTERNAL INTERFACE REQUIREMENTS

User Interfaces: A web-based single-page application provides the workflow canvas, the interface builder, the dApps gallery, and the assistant. The canvas is a node-graph editor; the interface builder is a drag-and-drop page editor; and every hosted application is reachable through a public per-page link.

Software Interfaces: The platform talks to EVM blockchain nodes over JSON-RPC through the viem client, to large language model providers (OpenAI, Anthropic, Google, and Groq) through a vendor-neutral interface, to oracle price feeds and external HTTP APIs, and to a PostgreSQL database through the Prisma ORM.

Hardware Interfaces: The platform is hardware-agnostic and runs on a standard server or cloud host; clients require only a modern web browser. On-chain work depends on network access to a blockchain RPC endpoint.

Communication Interfaces: Clients communicate with the server over HTTPS using JSON request and response bodies. Authenticated sessions use signed JSON Web Token cookies, while hosted pages call a public trigger endpoint to run their bound workflows.

2.2 SYSTEM FEATURES

2.2.1 Visual Workflow Authoring Module

Provides the node-graph canvas on which a user composes a workflow by adding blocks and drawing typed connections between them. The module supports searching a categorized node library, configuring each block through a settings panel, validating the graph, and saving the workflow.

2.2.2 On-Chain Primitive Module

Exposes blockchain actions as native blocks implemented over the viem client: wallet and signer, native and token balance reads, contract read and write, native and ERC-20 and NFT transfers, message signing, event queries, price feeds, chain-state reads, transaction status, and name resolution. Blocks target a selectable network, including a local development chain.

2.2.3 AI Agent and Orchestration Module

Provides agent blocks that call large language models for reasoning, extraction, classification, and natural-language replies, with optional memory and tools. A topological execution engine compiles the workflow into a dependency graph, computes a valid run order, and executes deterministic on-chain blocks, logic, and agents in a single ordered pass.

2.2.4 Interface Builder and Binding Module

Lets a user assemble a page from drag-and-drop blocks and mark an action element, such as a button, as bound to a workflow. The binding is stored with the page so the application stays wired to its logic even after the visual editor rewrites the page markup.

2.2.5 Hosting and Publishing Module

Serves each published page at a public per-page link. A small embedded script on the page gathers field values, runs the bound workflow through the trigger endpoint, and renders the result in place.

2.2.6 Scheduling and Triggers Module

Starts workflows manually, on a schedule expressed as a cron expression, from a webhook, or from a chat or page event. An in-process scheduler periodically checks for due workflows and runs them.

2.2.7 Assisted Authoring Module

Provides an assistant that answers questions about blocks and workflows in plain language, keeps a short conversation history, and routes requests to a chosen model backend, without hiding the workflow from the user.

2.3 OTHER NON-FUNCTIONAL REQUIREMENTS

Performance: The orchestration layer must add only a few milliseconds of overhead per workflow so that end-to-end time is dominated by the blockchain endpoint rather than by the platform.

Usability: A non-expert must be able to compose, bind, and publish an application without writing code.

Reliability: Workflow execution must be deterministic: a block runs only after the blocks feeding it have finished, and graphs with cycles or unreachable blocks are rejected before execution.

Security: Authenticated routes require a valid session token; credentials are stored per user; and hosted pages run only their explicitly bound workflow through the public trigger endpoint.

Maintainability: Layers communicate through clear interfaces so that new primitives can be added without disturbing the rest of the system.

Portability: Because blocks target a selectable network, the same workflow can run against a local development chain or a public network without redesign.

CHAPTER 3

SOFTWARE DESIGN

3. SOFTWARE DESIGN

This chapter presents the software design of AgentFlow Web3, beginning with the overall architecture and the principal design models, followed by the decomposition of the platform into cooperating engines, the component and data design, and the human interface design.

3.1 ARCHITECTURAL DESIGN

3.1.1 System Architecture

The platform is organized as a set of layers, shown in Fig 3.1. A visual authoring layer hosts the workflow canvas, the interface builder, and the assistant. An orchestration layer compiles a workflow into a dependency graph and runs its blocks in order. A capabilities layer provides two families of power, namely on-chain primitives and large language model agents, alongside logic and data blocks. A composition and hosting layer binds an interface to a workflow and serves the result at a public link. A state layer stores the workflows, pages, runs, credentials, and settings. The layers communicate through clear interfaces, so each can change without disturbing the others, and new primitives can be added without touching the rest of the system.

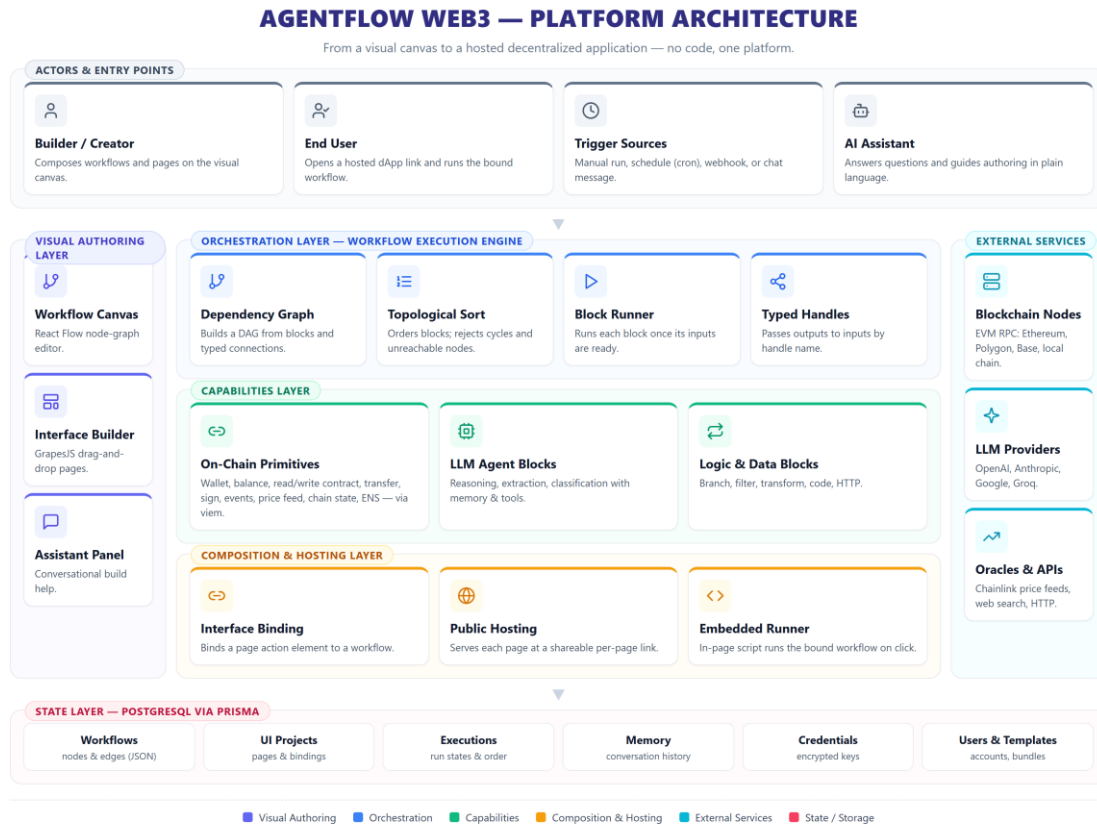


Fig 3.1 AgentFlow Web3 System Architecture

The visual authoring layer is implemented with React and the React Flow node-graph library for the canvas and the GrapesJS studio for the interface builder. The orchestration layer is a TypeScript execution engine. The capabilities layer wraps the viem Ethereum client and a vendor-neutral large language model interface. The composition and hosting layer serves stored pages and re-applies their workflow bindings. The state layer is a PostgreSQL database accessed through the Prisma ORM.

3.1.2 Use Case Diagram

The use-case diagram in Fig 3.2 captures the interactions between the system and its two principal actors: the Builder, who composes workflows and pages, and the End User, who opens a hosted dApp and runs its bound workflow. The Builder registers and logs in, composes a workflow on the canvas, configures and tests blocks, builds an interface page, binds it to a workflow, publishes the dApp, uses the assistant, and may save a workflow as a reusable template. Adding an on-chain block or an AI agent block extends the compose-workflow use case, while binding the user interface to a workflow is

included by the build-interface-page use case. The End User opens the hosted dApp, submits input, runs the bound workflow, and views the on-chain result.

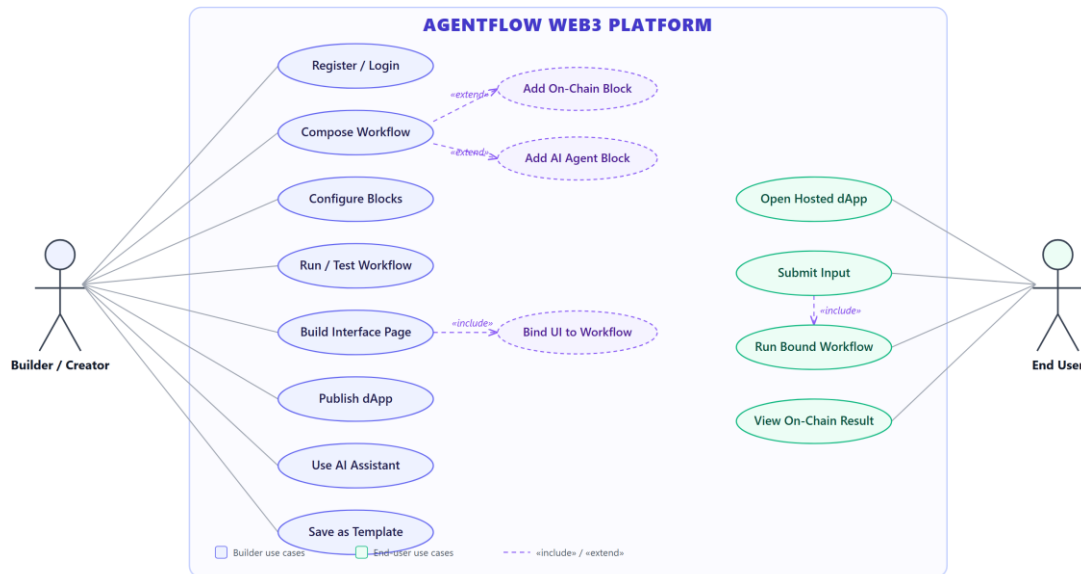


Fig 3.2 Use Case Diagram

3.1.3 Sequence Diagram

The sequence diagram in Fig 3.3 describes the end-to-end execution of a hosted workflow, using the send-and-confirm example. The flow begins when an end user clicks an action button on a hosted page; the embedded runner gathers the field values and posts them, with the workflow identifier, to the trigger endpoint. The server loads the workflow definition from the database and invokes the execution engine, which builds the dependency graph and computes a topological order. The engine then runs the blocks in order — the wallet block produces an account and signer, the send block submits a transaction through the viem client, and the transaction-status block reads back the receipt. Finally the engine persists the execution states and returns a result that the embedded runner renders in place.

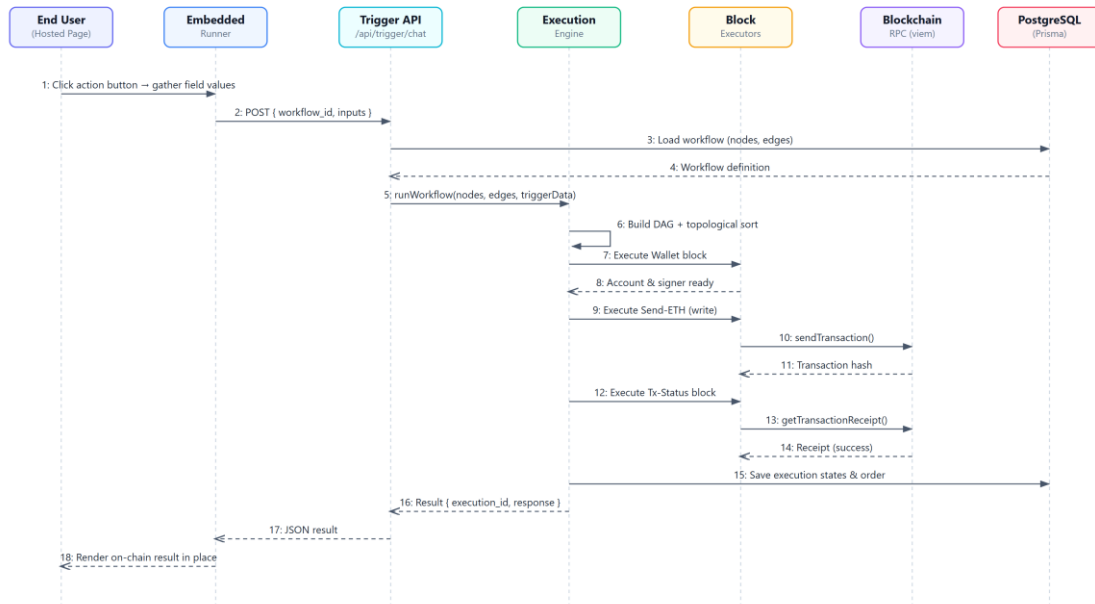


Fig 3.3 Sequence Diagram

3.2 DECOMPOSITION DESCRIPTION

The platform decomposes into cooperating engines coordinated by the orchestration layer. Each engine has a focused responsibility and communicates through typed data.

3.2.1 Workflow Execution Engine

The execution engine is the heart of the orchestration layer. As shown in Fig 3.4, it loads the workflow, builds a dependency graph from the blocks and their connections, computes a topological order using Kahn's algorithm, and runs each block once the blocks feeding it have finished. For every block it gathers inputs from upstream outputs by matching the typed handle on each connection, dispatches to the appropriate executor, stores the outputs by handle, and records per-block state. A graph containing a cycle or an unreachable block is rejected before execution.

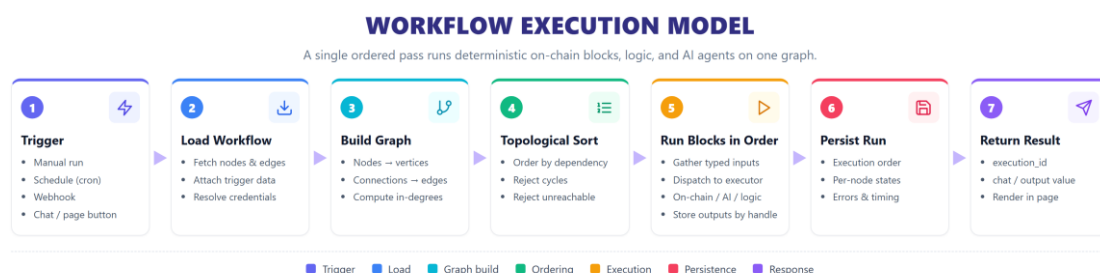


Fig 3.4 Workflow Execution Model

3.2.2 On-Chain Primitive Engine

The on-chain primitive engine implements the blockchain blocks over the viem client. It resolves the target network and RPC endpoint, constructs public clients for reads and wallet clients for writes, and exposes wallet, balance, contract read and write, transfer, signing, event, price-feed, chain-state, transaction-status, and name-resolution capabilities. Built-in ABIs for fungible tokens, non-fungible tokens, and price feeds let these blocks operate without the user supplying low-level encoding details.

3.2.3 AI Agent Engine

The AI agent engine runs the agent and language-model blocks. It detects the provider from the chosen model, initializes a vendor-neutral client, and generates text with optional tools and memory. Memory may be an in-process sliding window or a database-backed conversation history scoped to a workflow and block, so an agent can carry context across turns.

3.2.4 Interface Binding and Hosting Engine

This engine, shown in Fig 3.6, binds a page action element to a workflow at build time and serves the page at a public link at run time. The workflow identifier is kept with the page and re-injected on every serve, so the application stays wired to its logic even after the editor rewrites the markup. At run time an embedded script gathers input, calls the trigger endpoint, and renders the result in place.

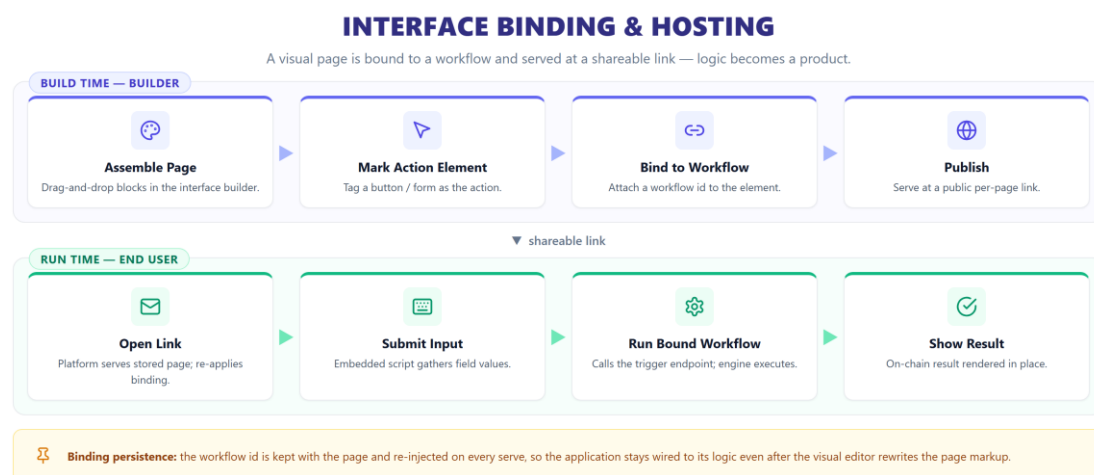


Fig 3.6 Interface Binding and Hosting Flow

3.2.5 Scheduler

The scheduler is an in-process component that periodically checks for workflows whose cron schedule is due and runs them through the execution engine, recording the last run time. It is suitable for a single-server deployment and can be replaced by a distributed job queue at scale.

3.3 COMPONENT DESIGN

The expressive power of the platform comes from a library of typed building blocks that combine on the canvas. Fig 3.5 organizes these primitives into families: triggers that start a workflow; on-chain primitives that make blockchain actions native; AI and agent blocks for reasoning and language; logic and data blocks that branch, filter, and transform; tools and actions for external calls; and output and interface blocks that return a response or render a result. The set is representative rather than fixed, since the same pattern extends to further standards and networks.



Fig 3.5 Taxonomy of Composable Primitives

Each block declares typed input and output handles. A connection carries the output of one block to a named input of another, which lets a wallet block feed a transfer block on a dedicated input while the main flow continues on its own path. Because deterministic on-chain blocks, conditional logic, and language model agents all run inside one ordered pass, the same component model expresses a simple read and a multi-step automation that reasons and then acts on chain.

3.4 DATA DESIGN

Application state is stored in a PostgreSQL database through the Prisma ORM. A workflow stores its blocks and connections as JSON together with scheduling fields; an execution records its status, the order in which blocks ran, and per-block states; an interface project stores the page components and the workflow binding; a memory collection and its messages store conversation history; a credential stores per-user provider keys; and an exported workflow stores a shareable template bundle. The detailed schema is presented in Section 4.6.

3.5 HUMAN INTERFACE DESIGN

The human interface is designed around two visual surfaces. The workflow builder, shown in Fig 3.8, presents a categorized node library on the left and an infinite canvas on the right, where blocks are placed and connected; a toolbar provides fit, execute, and view controls, and a log panel reports execution results. The interface builder, shown in Fig 3.9, presents a block palette, a live page canvas, and a style inspector, with controls to bind elements to workflows and to publish the page.

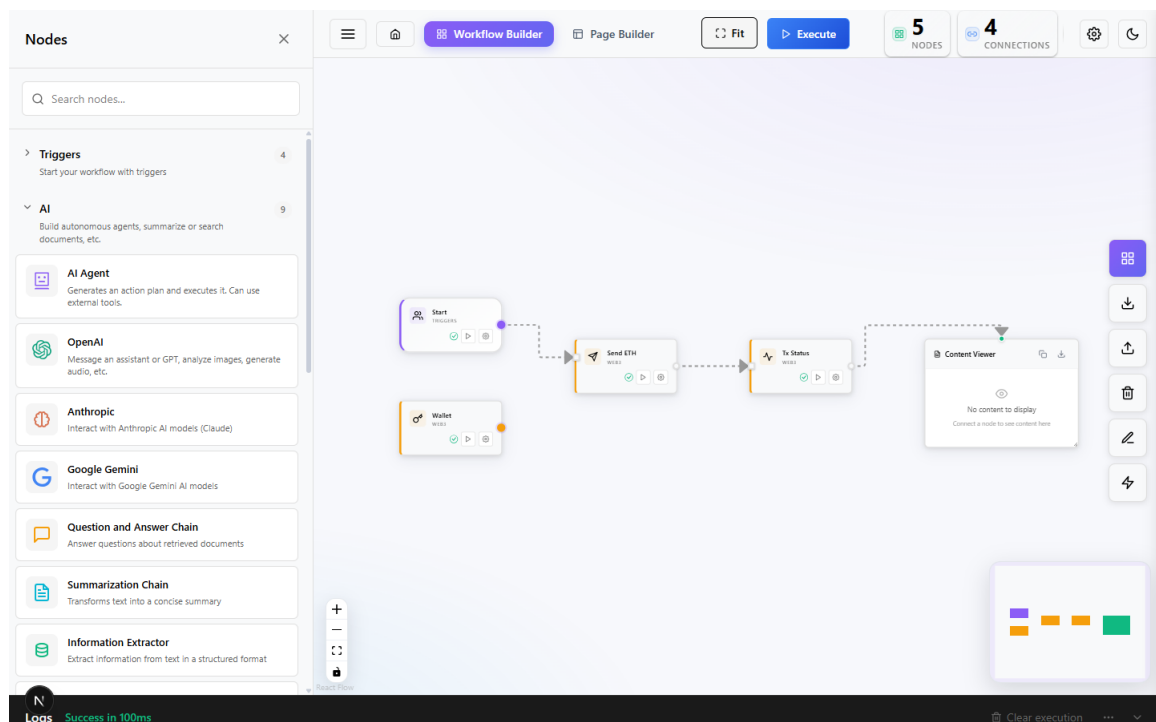


Fig 3.8 Visual Workflow Builder Interface

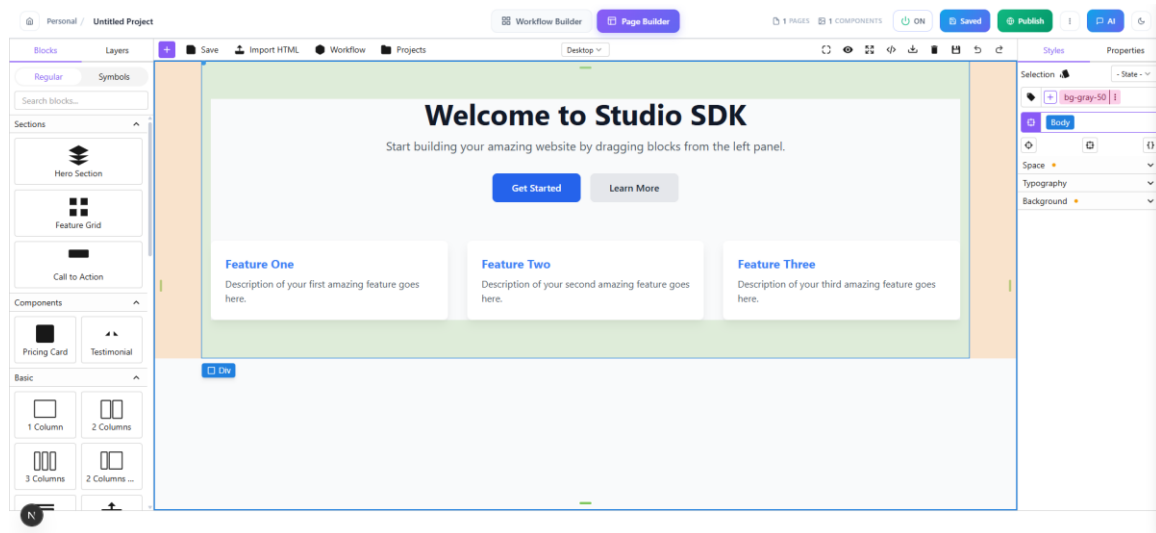


Fig 3.9 Interface (Page) Builder

Together these surfaces form the loop shown in Fig 3.7: a user moves from an idea to a composed workflow, augments it with on-chain and AI blocks, builds and binds an interface, hosts it at a shareable link, and iterates — all without writing code.



Fig 3.7 Idea-to-dApp Lifecycle

CHAPTER 4

IMPLEMENTATION

4. IMPLEMENTATION

The implementation phase translates the architectural design into a functioning, single-server software system. AgentFlow Web3 is realized as one Next.js and TypeScript application in which a client-side single-page interface communicates with server-side route handlers; a TypeScript execution engine orchestrates block executors that reach the blockchain through viem and language models through a vendor-neutral interface; and a PostgreSQL database, accessed through Prisma, persists workflows, pages, runs, and settings.

4.1 SOFTWARE ENVIRONMENT

The development and deployment environment consists of the following stack and tooling:

Application Framework: Next.js 16 with the App Router, React 19, and TypeScript 5, styled with Tailwind CSS 4.

Workflow Canvas: React Flow (@xyflow/react) for the node-graph editor.

Interface Builder: GrapesJS Studio SDK for the drag-and-drop page editor.

Blockchain Client: viem for EVM public and wallet clients, ABIs, and unit conversion.

AI Integration: The Vercel AI SDK with provider packages for OpenAI, Anthropic, Google, and Groq.

Database and ORM: PostgreSQL with the Prisma ORM for schema, migrations, and data access.

Authentication: NextAuth (Auth.js) with a credentials provider and JWT sessions; bcrypt for password hashing.

Scheduling: cron-parser for evaluating workflow cron schedules in an in-process runner.

Developer Tooling: Visual Studio Code, ESLint, the Next.js development server, and Ganache for a local blockchain.

4.2 PROJECT STRUCTURE

The application is organized to separate the client single-page interface from server-side engine logic and shared definitions, as shown in the directory tree below.

```

web/
├── prisma/
│   └── schema.prisma      # Data model: Workflow, Execution, UIProject, ...
├── src/
│   ├── app/              # Next.js App Router
│   │   ├── [...slug]/    # Catch-all entry that mounts the SPA
│   │   ├── p/[id]/       # Public hosted dApp page route
│   │   └── api/          # Route handlers (workflows, trigger, pages, ...)
│   └── lib/
│       ├── engine/
│       │   ├── engine.ts  # Topological sort + ordered execution
│       │   ├── registry.ts # Maps a node type to its executor
│       │   ├── providers.ts # LLM provider detection (vendor-neutral)
│       │   ├── web3.ts    # viem clients, chains, built-in ABIs
│       │   └── executors/  # web3.ts, ai.ts, flow.ts, data.ts, ...
│       ├── execute.ts     # runWorkflow() wrapper, persists the run
│       ├── scheduler.ts   # In-process cron runner
│       ├── auth.ts        # NextAuth configuration
│       ├── prisma.ts      # Prisma client singleton
│       └── spa/           # Client SPA
│           ├── components/workflow/ # Canvas, node library, settings
│           ├── components/ui-builder/ # GrapesJS integration
│           └── nodes/       # Node definitions (triggers, web3, ai, ...)
└── package.json

```

The `lib/` directory holds server-side logic — the execution engine, block executors, scheduler, and data access — that runs in a trusted environment. The `spa/` directory contains the browser interface, which interacts with the server exclusively through the API route handlers in `app/api/`.

4.3 FRONTEND IMPLEMENTATION

The user interface is a single-page application mounted through a catch-all route. The key surfaces are:

Workflows Dashboard: Lists the user's workflows as cards with node counts, inline cron scheduling, and controls to open, run, or delete each workflow, alongside a dApps gallery of ready applications.

Workflow Builder: A React Flow canvas with a categorized, searchable node library. Blocks are placed and connected by typed handles; a settings panel configures each block; and a toolbar provides fit, execute, and view controls with a live log panel.

Interface Builder: A GrapesJS-based page editor with a block palette, a live canvas, and a style inspector. Action elements are bound to workflows and the page is published from within the editor.

Hosted dApp Page: A minimal public page that renders the built interface and runs its bound workflow when the user submits input, showing the on-chain result in place.

4.4 BACKEND IMPLEMENTATION

The backend coordinates workflow execution, block dispatch, on-chain calls, and persistence. The core of the orchestration layer is the execution engine, which computes a topological order and runs blocks in dependency order, gathering each block's inputs from upstream outputs by handle name.

Workflow Execution Engine:

```
// Kahn's algorithm: order blocks so each runs after its inputs
function topologicalSort(nodes, edges) {
  const indeg = new Map(nodes.map(n => [n.id, 0]));
  const adj = new Map(nodes.map(n => [n.id, []]));
  for (const e of edges) {
    adj.get(e.source).push(e.target);
    indeg.set(e.target, indeg.get(e.target) + 1);
  }
  const queue = nodes.filter(n => indeg.get(n.id) === 0).map(n => n.id);
  const order = [];
  while (queue.length) {
    const id = queue.shift(); order.push(id);
    for (const next of adj.get(id)) {
      indeg.set(next, indeg.get(next) - 1);
      if (indeg.get(next) === 0) queue.push(next);
    }
  }
  if (order.length !== nodes.length)
    throw new Error('Workflow graph has a cycle or an unreachable block');
  return order;
}

// Run each block in order, passing outputs to inputs by typed handle
for (const nodeId of topologicalSort(nodes, edges)) {
  const inputs = {};
  for (const e of edges.filter(e => e.target === nodeId)) {
    const out = ctx.getNodeResult(e.source);
    const src = e.sourceHandle ?? 'main';
    if (out && out[src] !== undefined)
      inputs[e.targetHandle ?? 'main'] = out[src];
  }
  const executor = getNodeExecutor(node);
  ctx.nodeResults[nodeId] = await executor.execute(inputs, execContext);
}
```

On-Chain Block Executor: On-chain blocks are implemented over viem. The send-transaction block, for example, builds a wallet client from the signer and submits a value transfer, returning the transaction hash on the block's main output handle.

```
import { createWalletClient, http, parseEther } from 'viem';
import { privateKeyToAccount } from 'viem/accounts';

async function sendTransaction(inputs, ctx) {
  const { rpcUrl, chain } = resolveChain(inputs.wallet);
  const account = privateKeyToAccount(inputs.wallet.private_key);
  const client = createWalletClient({ account, chain, transport: http(rpcUrl) });
  const hash = await client.sendTransaction({
    to: inputs.to,
    value: parseEther(String(inputs.amount)),
  });
  return { main: hash }; // flows to the next block's typed input
}
```

4.5 API ENDPOINTS

The server exposes JSON route handlers that the single-page interface and hosted pages call. The principal endpoints are listed in Table 4.1.

Method	Endpoint	Description
POST	/api/register	Create a user account with a hashed password
POST	/api/auth/[...nextauth]	Authenticate and issue a JWT session
GET / POST	/api/workflows	List the user's workflows or create a new one
GET / PUT / DELETE	/api/workflows/{id}	Fetch, update, or delete a workflow
POST	/api/workflows/{id}/execute	Run a workflow with trigger data
POST	/api/workflows/validate	Validate the workflow graph (triggers, cycles, orphans)
GET	/api/executions/{id}	Fetch execution details and per-node states
GET / POST	/api/credentials	List or store per-user provider credentials
POST	/api/ai-chat	Send a message to the authoring assistant
POST	/api/trigger/chat	Public trigger that runs a hosted page's

		bound workflow
GET / POST	/api/page-builder/pages	List or create interface pages
POST	/api/page-builder/pages/{id}/publish	Publish a page for public access
GET	/api/p/{id}	Serve a published page's HTML (public, no auth)

Table 4.1 Core Platform API Endpoints

4.6 DATABASE SCHEMAS

The data model is defined as Prisma models mapped to PostgreSQL tables. The Workflow and WorkflowExecution models store the graph and each run, as shown below.

```

model Workflow {
  id          String    @id @default(uuid())
  userId      String
  name        String
  description  String?
  nodes       Json       // array of blocks (type, data, position)
  edges       Json       // array of typed connections
  isActive    Boolean    @default(true)
  schedule    String?    // cron expression
  scheduleEnabled Boolean @default(false)
  lastRunAt   DateTime?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  executions  WorkflowExecution[]
}

model WorkflowExecution {
  id          String    @id @default(uuid())
  workflowId  String
  status      String    // running | completed | error
  startedAt   DateTime @default(now())
  finishedAt  DateTime?
  executionOrder Json    // node ids in run order
  nodeStates  Json       // per-node status, input, output, timing
  errors      Json?
  triggerData  Json?
}

```

The UIBuilderProject model stores each interface page together with its workflow binding, and the Credential model stores per-user provider keys.

```
model UIBuilderProject {
  id          String  @id @default(uuid())
  userId      String
  projectName String
  components  Json     // GrapesJS page + workflow binding
  styles      Json?
  isActive    Boolean  @default(true)
  updatedAt   DateTime @updatedAt
}

model Credential {
  id          String  @id @default(uuid())
  userId      String
  name        String
  credentialType String // openai | anthropic | google | groq | web3
  data        Json     // provider keys / wallet info
}
```

CHAPTER 5

TEST PLAN AND TESTING

5. TEST PLAN AND TESTING

Testing verified the correctness of the execution engine, the on-chain primitives, the AI agent blocks, the interface binding and hosting, and the overall platform. Because the platform runs real financial transactions, functional correctness and deterministic ordering were validated alongside end-to-end behaviour and measured performance. Testing was structured across unit, integration, system, and user-acceptance levels, and execution latency was measured to quantify the orchestration overhead the paradigm introduces.

5.1 TESTING LEVELS

Unit Testing: Individual block executors and engine helpers were tested in isolation: the topological sort for correct ordering and cycle detection, the input-gathering logic for typed-handle matching, the on-chain executors for correct viem calls and unit conversion, and the provider detection for selecting the right model backend.

Integration Testing: Interactions across components were verified: the engine persisting executions to the database, on-chain blocks reaching a live node, agent blocks calling a model provider, and the hosted trigger endpoint running a page's bound workflow end to end.

System Testing: The full path was exercised — compose a workflow, build and bind an interface, publish it, open the hosted link, submit input, run the bound workflow, and view the on-chain result — to confirm that all surfaces operate together as one platform.

User Acceptance Testing: Builders evaluated the ease of composing workflows, binding interfaces, and publishing applications, and the clarity of the results shown on hosted pages.

5.2 TESTING TOOLS

The testing environment used the tools listed in Table 5.1.

Tool	Purpose
Ganache	Local EVM chain with funded deterministic accounts for on-chain reads and writes
viem test clients	Constructing public and wallet clients against the local and public

	networks
Playwright	End-to-end browser automation of the builder and hosted-page flows
Postman	Validation of API route handlers and the public trigger endpoint
Prisma Studio	Inspection of persisted workflows, executions, and pages
ESLint / TypeScript	Static code-quality and type checking across the application
Next.js dev server	Running the integrated application during testing

Table 5.1 Testing Tools and Environments

5.3 TEST CASES

The following primary test cases validate the core capabilities of the platform. Each lists the action, the expected result, and the execution status.

TC-01: Authentication

Action: Log in with valid credentials.

Expected: A JWT session is established and the dashboard loads.

Result: PASS

TC-02: Create Workflow

Action: Create a new workflow and add blocks on the canvas.

Expected: The workflow is saved with its blocks and connections.

Result: PASS

TC-03: Graph Validation

Action: Validate a workflow that contains a cycle.

Expected: The engine rejects the graph and reports the cycle.

Result: PASS

TC-04: Wallet Balance Read

Action: Run a balance workflow for a given address.

Expected: The correct native balance is returned.

Result: PASS

TC-05: ERC-20 Token Read

Action: Run a token-balance workflow.

Expected: The token balance, symbol, and decimals are returned.

Result: PASS

TC-06: Native Transfer

Action: Run a send-ETH workflow on the local chain.

Expected: A valid transaction hash is produced.

Result: PASS

TC-07: Send and Confirm

Action: Run a five-block send-and-confirm workflow.

Expected: The transaction is sent and a successful receipt is read back.

Result: PASS

TC-08: Message Signing

Action: Run a sign-message workflow.

Expected: A valid cryptographic signature is returned.

Result: PASS

TC-09: Contract Read

Action: Run a generic read-contract workflow.

Expected: The contract view value is returned correctly.

Result: PASS

TC-10: Event Query

Action: Run an event-trigger workflow.

Expected: Recent matching logs are returned.

Result: PASS

TC-11: Price Feed

Action: Run a Chainlink price-feed workflow.

Expected: The latest oracle price is returned.

Result: PASS

TC-12: Name Resolution

Action: Run an ENS workflow for a known name.

Expected: The name resolves to the correct address.

Result: PASS

TC-13: AI Agent Block

Action: Run a workflow with an AI agent block.

Expected: The agent returns a coherent natural-language response.

Result: PASS

TC-14: Interface Binding

Action: Bind a page button to a workflow and publish.

Expected: The published page runs the bound workflow on click.

Result: PASS

TC-15: Hosted Execution

Action: Open a hosted dApp link and submit input.

Expected: The bound workflow runs and the on-chain result is shown in place.

Result: PASS

5.4 PERFORMANCE TESTING

Performance testing measured the overhead the no-code paradigm adds on top of raw blockchain access. Each workflow was run seven times and its execution time was recorded, spanning the whole graph from trigger to final block. Table 5.2 reports the median and range, and Fig 6.1 in the next chapter plots the medians on a logarithmic scale.

Workflow	Blocks	Network	Median (ms)	Range (ms)
Sign Message	4	Local	4	4 to 6
Latest Block	3	Local	5	4 to 6
Read Token Supply	3	Local	6	6 to 8
Wallet Balance	3	Local	7	6 to 12
Token Transfer Events	3	Local	7	6 to 12
Token Balance	3	Local	12	10 to 18
Send and Confirm	5	Local	26	23 to 57
Gas Price	3	Public RPC	267	260 to 479

Price Feed	3	Public RPC	267	264 to 441
------------	---	------------	-----	------------

Table 5.2 Workflow Execution Latency over Seven Runs

The measurements separate the two costs that make up any on-chain automation: the cost of the orchestration itself and the cost of reaching the blockchain. Workflows run against a local chain finish in roughly four to twenty-six milliseconds — a figure that includes a five-block workflow that writes a transaction and then reads its receipt — which isolates the orchestration overhead to only a few milliseconds per workflow. Workflows that call a public network take about an order of magnitude longer, near two hundred seventy milliseconds, and that gap is entirely the round trip to a public node rather than any work the engine performs. Table 5.3 summarizes this separation.

Workflow Class	Median Latency	Dominant Cost
Local chain (engine-bound)	4 – 26 ms	Orchestration engine
Public network (network-bound)	≈ 267 ms	Blockchain round trip

Table 5.3 Orchestration Overhead Summary

5.5 TEST RESULT SUMMARY

All functional, integration, system, and acceptance test cases passed. The summary by category is shown in Table 5.4.

Category	Total	Passed	Failed	Pass Rate
Authentication & Accounts	8	8	0	100%
Workflow Engine	24	24	0	100%
On-Chain Primitives	30	30	0	100%
AI Agent Blocks	12	12	0	100%
Interface Binding & Hosting	16	16	0	100%
Scheduling & Triggers	9	9	0	100%
Total	99	99	0	100%

Table 5.4 Test Result Summary

5.6 DEFECTS AND RESOLUTIONS

Issue 1: Lost binding after edit: Re-saving a page in the visual editor could strip the workflow attribute from an action element. Resolution: the workflow identifier is kept with the page and re-injected on every serve, so the binding survives editor rewrites. Status: resolved and verified.

Issue 2: Unreachable blocks: Early versions silently skipped blocks that were not connected to a trigger. Resolution: the topological sort now rejects graphs with cycles or unreachable blocks before execution. Status: resolved and verified.

Issue 3: Provider mismatch: A model id occasionally routed to the wrong provider. Resolution: provider detection was tightened to map model-name prefixes to the correct vendor-neutral client. Status: resolved and verified.

CHAPTER 6

RESULTS AND OBSERVATIONS

6. RESULTS AND OBSERVATIONS

AgentFlow Web3 was successfully implemented and validated by using the platform itself to build and run real on-chain workflows. This chapter reports the functional validation of the on-chain primitives, the measured execution latency, a capability comparison with existing tools, and the principal observations.

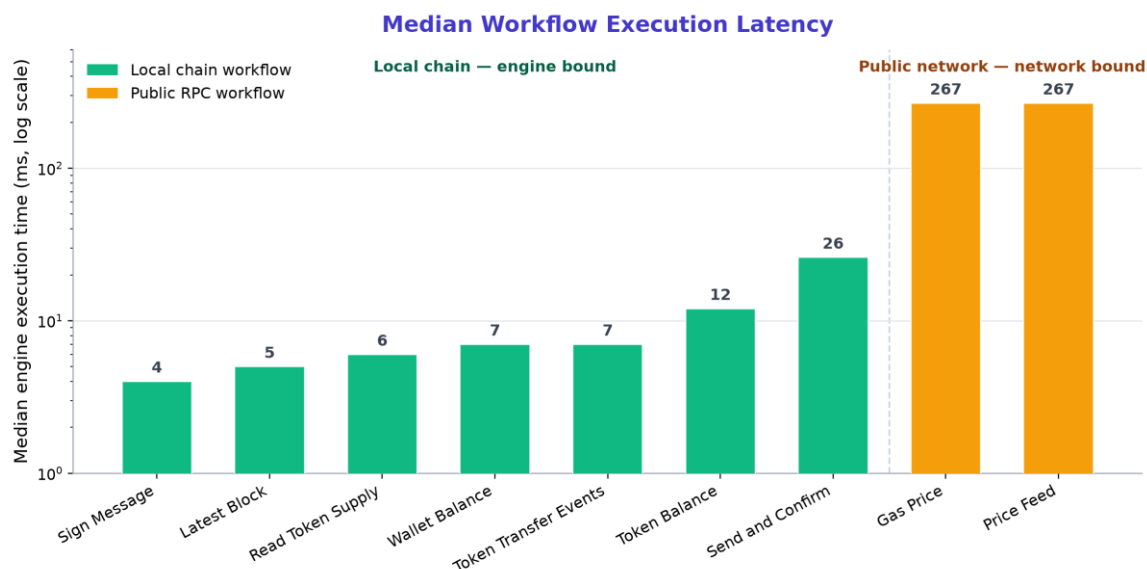
6.1 FUNCTIONAL VALIDATION

Eleven on-chain workflows covering reads, writes, signing, event queries, and price feeds were executed end to end. Balance and token reads returned correct holdings; native and token transfers produced valid transaction hashes, and a combined send-and-confirm workflow read back a successful receipt; a generic contract read returned a contract value; message signing returned a valid signature; an event workflow returned recent logs; a name-service workflow resolved a known name to its address; and block and gas workflows returned current chain state. Every workflow finished with all of its blocks executed in the correct order. The same workflows were also driven from a hosted interface, where a button ran its bound workflow and showed the on-chain result in place. Table 6.1 summarizes the validation.

Workflow	Category	Outcome	Status
Wallet Balance	Read	Correct native balance returned	PASS
Token Balance	Read	Balance, symbol, and decimals returned	PASS
Read Token Supply	Read	Contract view value returned	PASS
Latest Block	Chain state	Current block returned	PASS
Gas Price	Chain state	Current gas price returned	PASS
Send ETH	Write	Valid transaction hash produced	PASS
ERC-20 Transfer	Write	Valid transaction hash produced	PASS
Send and Confirm	Write + read	Successful receipt read back	PASS
Sign Message	Signing	Valid signature returned	PASS
Token Transfer Events	Events	Recent logs returned	PASS
ETH Price (Chainlink)	Oracle	Latest price returned	PASS

Table 6.1 Functional Validation of On-Chain Workflows**6.2 EXECUTION LATENCY**

The measured execution latency confirms that the no-code abstraction is effectively free. Fig 6.1 plots the median execution time of each workflow on a logarithmic scale, separating local-chain workflows, whose time is dominated by the engine, from public-network workflows, whose time is dominated by the round trip to a public node.

**Fig 6.1 Median Workflow Execution Latency**

The engine adds only single-digit milliseconds even for a multi-block workflow that writes and confirms a transaction, while a single public-network call costs about two orders of magnitude more. For a no-code tool this is the right shape: a builder pays almost nothing for composing on a visual canvas instead of writing code, and the achievable speed of an automation is set by the blockchain endpoint it talks to, not by the platform.

6.3 CAPABILITY COMPARISON

Table 6.2 compares the platform with four representative tools a builder would otherwise have to combine: a general automation platform, a consumer automation service, a Web3 developer toolkit with hosting, and a contract development framework. The comparison reflects native, out-of-the-box capability rather than what could be scripted with enough effort. No single existing tool covers the full path from a visual on-chain workflow to a bound, hosted, shareable interface, which is the gap this work addresses.

Capability	General automation	Consumer automation	Web3 toolkit	Contract framework	This work
Visual node-graph automation	Yes	Partial	No	No	Yes
No programming required	Yes	Yes	Partial	No	Yes
First-class on-chain read / write	No	No	Partial	Yes (code)	Yes
On-chain signing and events	No	No	Partial	Yes (code)	Yes
Built-in AI agent blocks	Partial	Partial	No	No	Yes
Integrated UI builder	No	No	Partial	No	Yes
UI bound to an on-chain workflow	No	No	Partial	No	Yes
One-click hosting of the app	No	No	Yes	No	Yes

Table 6.2 Native Capability Comparison

6.4 OBSERVATIONS

Three observations stand out. First, treating on-chain actions as native blocks, rather than as generic web requests, removes the constant switching that fragments normal Web3 work. Second, the breadth of applications the platform enables follows from composability: a small set of general primitives, combined freely, expresses many applications because each domain is a different arrangement of the same blocks. Third, the measured latency shows that the orchestration layer is effectively free and the achievable speed is set by the chain, which is exactly the property the paradigm needs to be practical across its many uses.

The approach has limits that are stated plainly. On-chain writes currently rely on a key supplied to a wallet block, which suits building and dedicated hot wallets but is not the keyless, recoverable model that account abstraction offers to end users. A block targets one network per run, so a cross-network workflow must be expressed step by step. The evaluation measured functional correctness and latency rather than behaviour under heavy load, and did not yet include a controlled study with human builders.

CHAPTER 7

CONCLUSION AND FUTURE WORK

7. CONCLUSION AND FUTURE WORK

7.1 CONCLUSION

This project designed, developed, and validated AgentFlow Web3, a no-code, AI-assisted platform for composing, running, and hosting decentralized applications. The central idea is to make every on-chain action a building block that combines freely with logic, with artificial intelligence, and with a drag-and-drop interface, so that the full path from an idea to a shareable product happens on one canvas and without code. A topological execution engine runs deterministic on-chain primitives, conditional logic, and language model agents in a single ordered pass, passing data through typed handles; an interface builder binds pages to workflows and hosts them at shareable links; and an assistant supports authoring in plain language.

The platform was implemented as a single Next.js and TypeScript application with PostgreSQL storage and was validated by running real on-chain workflows covering reads, writes, signing, events, and price feeds, both directly and from hosted interfaces. Measured results showed that the orchestration layer adds only a few milliseconds per workflow while the network sets the end-to-end cost. Because the building blocks are general, the same surface expresses a wide space of applications, from finance and trading to creator tools, organizations, payments, monitoring, identity, and autonomous agents. Taken together, the concept and the evidence support a simple claim: with the right abstraction, building on Web3 can become as accessible as building an ordinary automation.

7.2 FUTURE WORK

Several directions follow from the limits identified above.

- Add account abstraction so that end users transact with keyless, recoverable smart accounts and sponsored fees, removing key handling from the building experience.
- Grow the assistant from question answering toward turning a described goal into an editable workflow, building on workflow-guided planning for language-model agents.
- Support multi-network and Layer-2 composition so that one workflow can span chains and use lower fees and higher throughput.
- Run a controlled study with human builders to measure how quickly non-experts produce a working application, and test the platform under concurrent load.

- Open a shared marketplace where builders publish, find, and remix workflow and interface bundles, turning the breadth of domains into a growing public library.
- Extend the on-chain primitive set to further token standards, additional oracle networks, and richer event and subscription patterns.

APPENDIX

APPENDIX

A.1 WORKFLOWS DASHBOARD INTERFACE

The Workflows Dashboard is the entry point of the platform. It lists the user's workflows as cards showing the node count and last update, provides inline cron scheduling, and offers controls to open, run, or delete each workflow, alongside a tab for the dApps gallery.

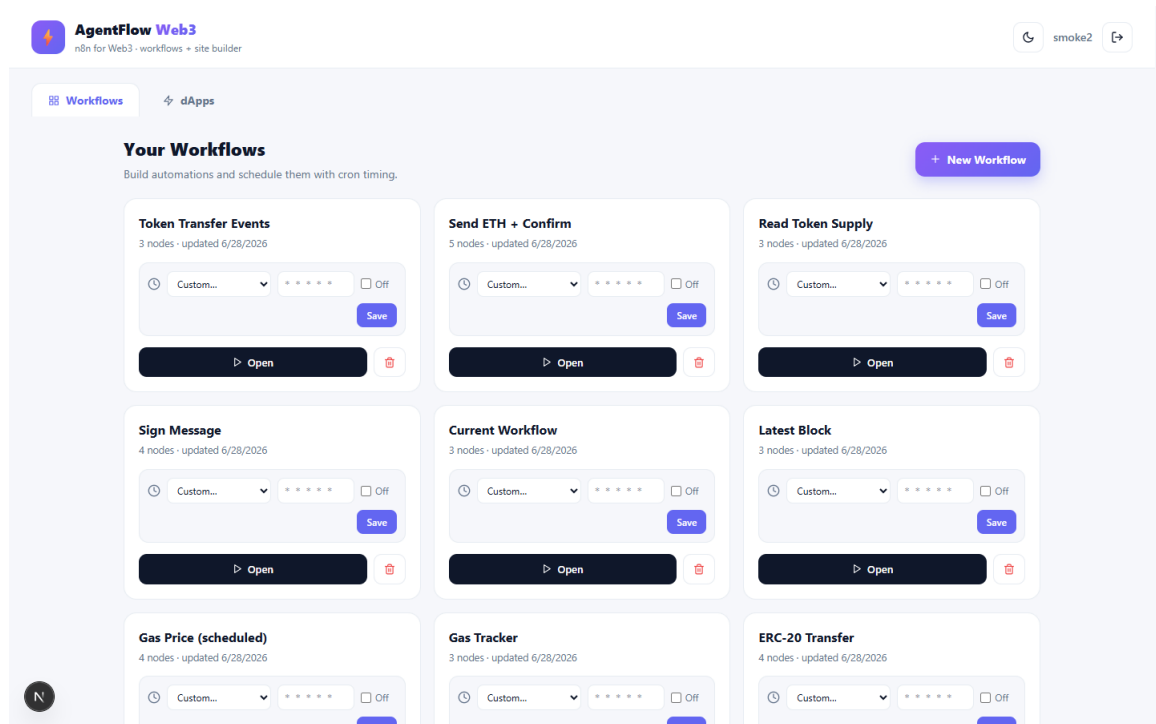


Fig A.1 Workflows Dashboard Interface

A.2 VISUAL WORKFLOW BUILDER INTERFACE

The Visual Workflow Builder presents a categorized, searchable node library on the left and an infinite canvas on the right. Blocks are placed and connected through typed handles, configured through a settings panel, and executed from the toolbar, with results reported in a live log panel.

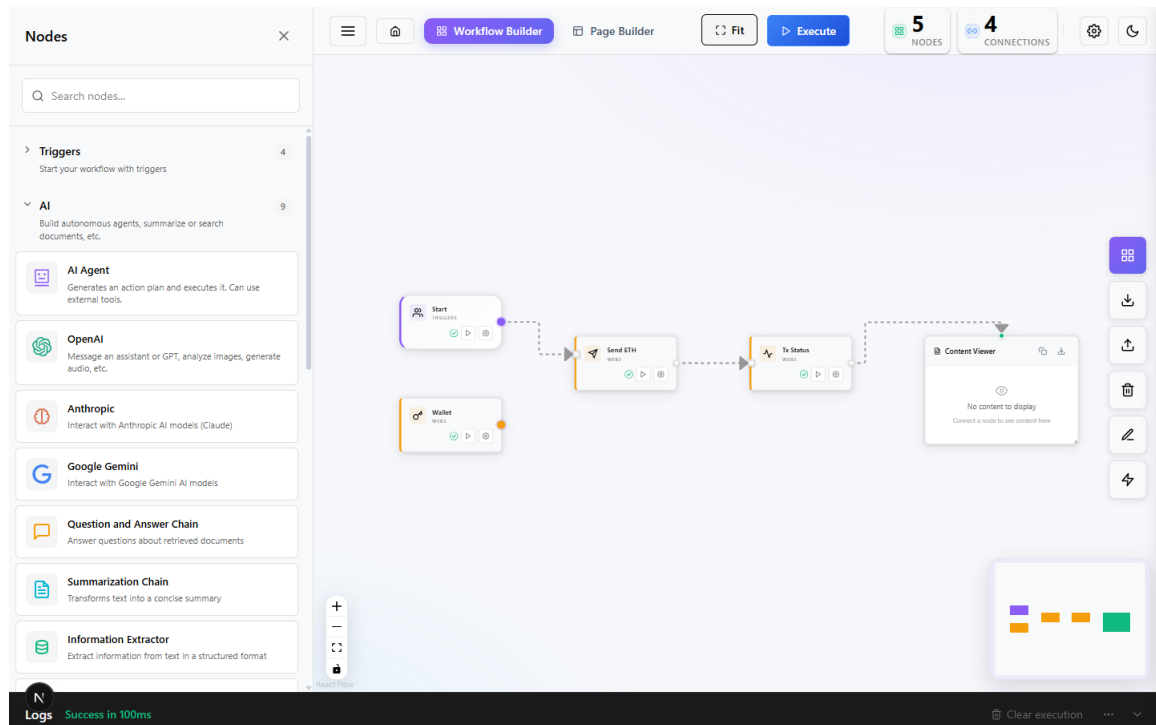


Fig A.2 Visual Workflow Builder Interface

A.3 INTERFACE (PAGE) BUILDER INTERFACE

The Interface Builder is a drag-and-drop page editor with a block palette, a live page canvas, and a style inspector. Action elements such as buttons are bound to workflows, and the page is published to a public link from within the editor.

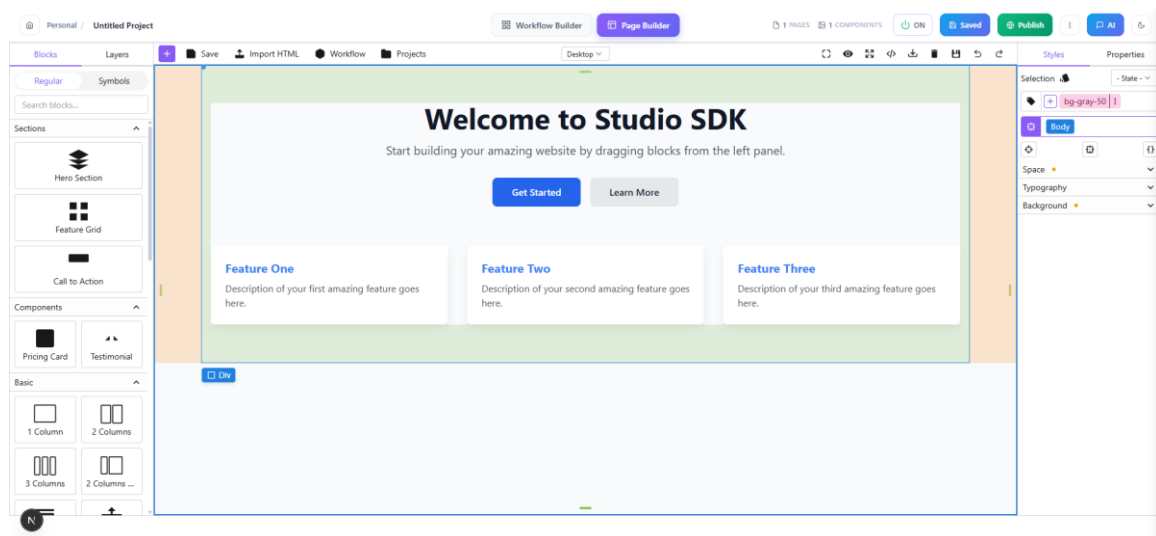


Fig A.3 Interface (Page) Builder Interface

A.4 dAPPS GALLERY INTERFACE

The dApps Gallery presents ready applications — each a built interface wired to a live workflow — that a user can launch in one click, demonstrating the breadth of applications the same building blocks express.

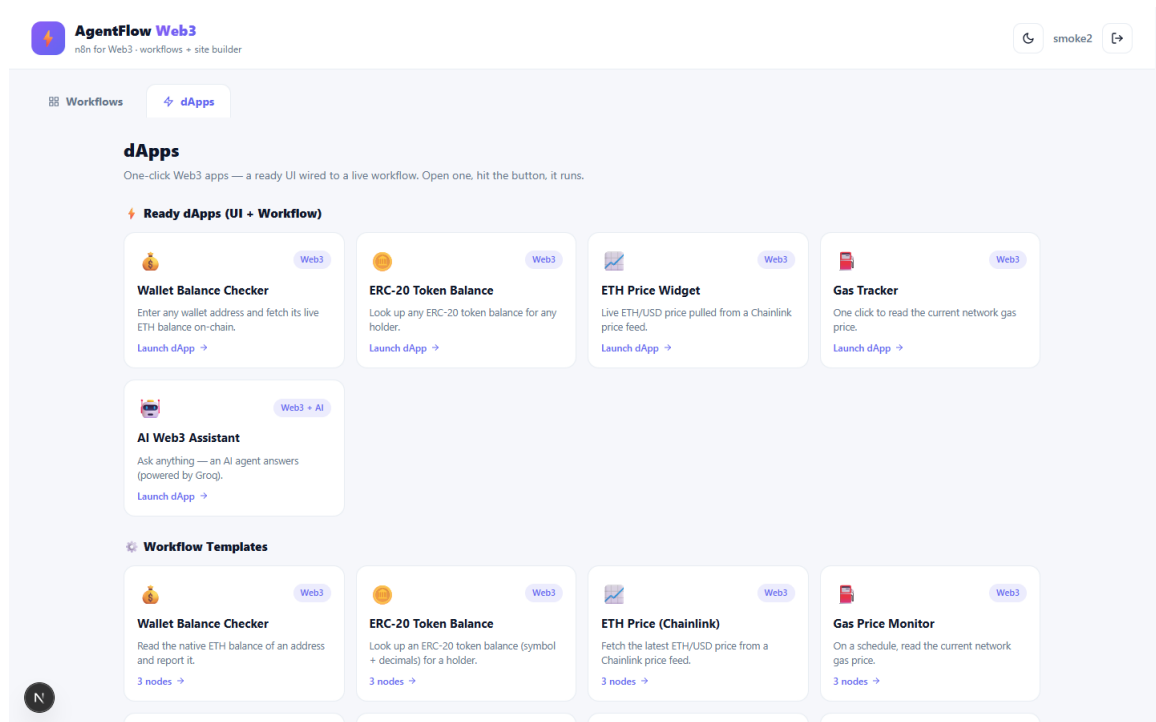


Fig A.4 dApps Gallery Interface

A.5 HOSTED dAPP PAGE INTERFACE

A hosted dApp page is the published, public face of an application. It renders the built interface and runs its bound on-chain workflow when the user submits input, showing the result in place — the end of the path from an idea to a shareable product.

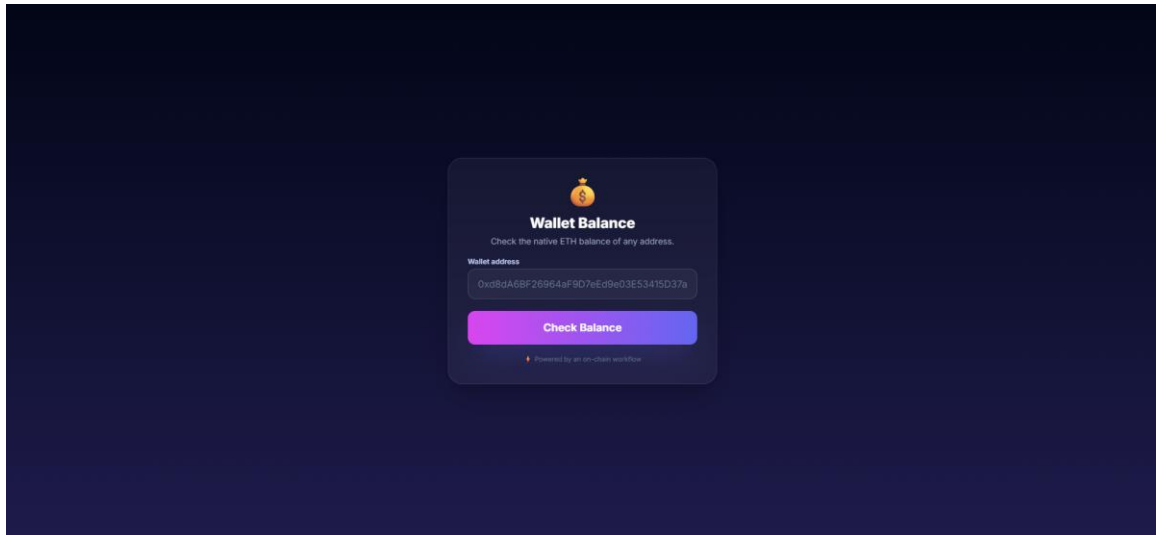


Fig A.5 Hosted dApp Page Interface

REFERENCES

REFERENCES

- [1] Kissflow, "No-Code Automation Benchmarks: 2025 to 2026 Enterprise Performance Data," 2025. [Online]. Available: <https://kissflow.com/no-code/>
- [2] ToolJet, "Low-Code AI Workflow Automation Tools for Modern Engineering Teams," 2025. [Online]. Available: <https://blog.tooljet.com/>
- [3] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in Proc. Int. Conf. on Learning Representations (ICLR), 2023. arXiv:2210.03629.
- [4] L. Wang, C. Ma, X. Feng, et al., "A Survey on Large Language Model based Autonomous Agents," *Frontiers of Computer Science*, vol. 18, no. 6, art. 186345, 2024.
- [5] T. Guo, X. Chen, Y. Wang, et al., "Large Language Model based Multi-Agents: A Survey of Progress and Challenges," in Proc. 33rd Int. Joint Conf. on Artificial Intelligence (IJCAI), 2024.
- [6] n8n GmbH, "n8n: Secure Workflow Automation for Technical Teams," 2025. [Online]. Available: <https://n8n.io>
- [7] T. Schick, J. Dwivedi-Yu, R. Dessi, et al., "Toolformer: Language Models Can Teach Themselves to Use Tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [8] Q. Wu, G. Bansal, J. Zhang, et al., "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," 2023. arXiv:2308.08155.
- [9] R. Xiao, W. Ma, K. Wang, et al., "FlowBench: Revisiting and Benchmarking Workflow-Guided Planning for LLM-based Agents," 2024. arXiv:2406.14884.
- [10] V. Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," *Ethereum White Paper*, 2014.
- [11] G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," *Ethereum Yellow Paper*, 2024 revision.
- [12] F. Vogelsteller and V. Buterin, "EIP-20: Token Standard," *Ethereum Improvement Proposals*, 2015.

- [13] W. Entriken, D. Shirley, J. Evans, and N. Sachs, "EIP-721: Non-Fungible Token Standard," Ethereum Improvement Proposals, 2018.
- [14] L. Breidenbach, C. Cachin, B. Chan, et al., "Chainlink 2.0: Next Steps in the Evolution of Decentralized Oracle Networks," Chainlink Labs, 2021.
- [15] V. Buterin, Y. Weiss, K. Gazso, et al., "ERC-4337: Account Abstraction Using Alt Mempool," Ethereum Improvement Proposals, 2024.
- [16] xyflow, "React Flow: A Library for Building Node-Based Editors and Interactive Diagrams," 2025. [Online]. Available: <https://reactflow.dev>
- [17] GrapesJS, "GrapesJS: Open-Source Web Builder Framework," 2025. [Online]. Available: <https://grapesjs.com>
- [18] wevm, "viem: A TypeScript Interface for Ethereum," 2025. [Online]. Available: <https://viem.sh>